



Multi-Dimensional Arrays

Advanced Data Structures in C Programming



2D Arrays



3D Arrays



Chess Example





Introduction to Multi-Dimensional Arrays



C programming language allows **multidimensional arrays**. There is no limit to the number of subscripts (or dimensions) an array can have.

General Form:

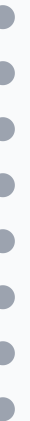
```
type name[size1][size2]...[sizeN];
```

◆ Visualization

While humans cannot easily visualize objects with more than three dimensions, representing multi-dimensional arrays presents no problem to computers.

◆ Memory Layout

Arrays with more than one dimension are called multi-dimensional arrays.





Chessboard Example



Now you are writing a chess-playing program. A chessboard is an **8-by-8 grid**. What data structure would you use to represent it?

Solution: Two-Dimensional Array

You could use an array that has a chessboard-like structure, i.e., a two-dimensional array to store positions of chess pieces.



Indexing

Two-dimensional arrays use two indices to pinpoint an individual element of an array.



Algebraic Notation

This is very similar to what is called "algebraic notation", commonly used in chess circles to record games and chess problems.

```
int three_dim[5][10][4];
```



Key Point: Remember, C arrays are zero-based, so indices on each side of the chessboard array run 0 through 7, rather than 1 through 8.





Multi-Dimensional Arrays Declaration



To declare an array of two dimensions as follows:

Syntax:

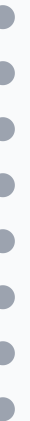
```
datatype array_name[size1][size2];
```

◆ Parameters

variable_type is the name of some type, such as int. size1 and size2 are the sizes of the array's first and second dimensions, respectively.

◆ Memory Requirements

An $m \times n$ array will have $m*n$ elements and will require $m*n*\text{element size}$ bytes of storage.





Initialization of Two-Dimensional Arrays



The simplest form of multidimensional array is a **two-dimensional array**. A two-dimensional array is, in essence, a list of one-dimensional arrays.

Syntax:

```
type arrayName[x][y];
```



Row-wise Storage

The elements of a two-dimensional array are stored row wise.



Element Access

To access an element, specify both row and column indices.





Initialization Examples



The initial values must appear in the order in which they will be assigned to individual array elements, enclosed within braces and separated by commas.

```
int table[2][3] = {1,2,3,4,5,6};
```

◆ Row-wise Assignment

It means that element `table[0][0] = 1`; `table[0][1] = 2`; etc.

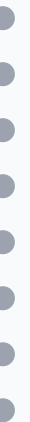
◆ Group Initialization

The value within innermost braces will be assigned to those array elements whose last subscript changes most rapidly.

```
int table[2][3] = { {1,2,3}, {4,5,6} };
```



Key Point: If there are few remaining values in a row, they will be assigned zeros. The number of values cannot exceed the defined row size.





Memory Layout and Storage



Understanding how multi-dimensional arrays are stored in memory is crucial for efficient programming.

Row-major Order

C stores multi-dimensional arrays in row-major order, meaning elements of each row are stored contiguously.

```
int table[2][3] = {1,2,3,4,5,6};
```

Memory: 1,2,3,4,5,6 followed by 4,5,6

Column-major Order

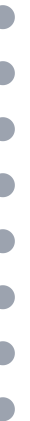
Some languages use column-major order, but C uses row-major order by default.

```
int table[2][3] = {1,2,3,4,5,6};
```

Memory: 1,2,3,4,5,6 followed by 4,5,6



Memory Calculation: For a 2×3 array with int elements (4 bytes each), total memory = $2 \times 3 \times 4 = 24$ bytes.





Practical Applications



Multi-dimensional arrays are used in various real-world applications.

Matrix Operations

Mathematical operations on matrices, such as addition, multiplication, and transposition.

```
int result[3][3];
for(i = 0; i < 3; i++) {
    for(j = 0; j < 3; j++) {
        result[i][j] = 0;
        for(k = 0; k < 3; k++) {
            result[i][j] += matrix1[i][k] * matrix2[k][j];
        }
    }
}
```

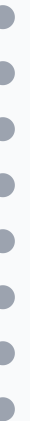
Image Processing

2D arrays to represent pixel data in images for filtering, transformation, and analysis.

```
int image[height][width];
for(y = 0; y < height; y++) {
    for(x = 0; x < width; x++) {
        image[y][x] = processed_pixel_value;
    }
}
```

Game Development

Board games, puzzles, and simulations often use multi-dimensional arrays to represent game states.




```
int board[8][8];  
board[row][col] = piece_type;
```



Summary



Multi-Dimensional Arrays

Arrays with multiple dimensions for complex data structures



Initialization

Row-wise assignment with braces and commas



Memory Layout

Row-major order storage in contiguous memory



Multi-dimensional arrays provide powerful tools for representing complex data structures in C programming!



